

VERIFORM

Verifiable AI Agents with TEE-Backed Decision Receipts

Proof, not blind trust.

MASTER PROJECT REPORT

Problem • Existing Solutions • Approach • Architecture
Technology • Evidence • Completed and Pending Work

Version 1.0 | 13 August 2026

Sakthi Sundaram R | github.com/Sakthi-Sundaram-R/Veriform

77 tests passing | CI green | Live on Ethereum Sepolia | Proven on real Intel TDX silicon

Contents

1	Executive Summary
2	Problem Statement
3	Background: TEEs and Remote Attestation
4	Existing Solutions vs Veriform
5	Solution Approach - The Decision Receipt
6	System Architecture
7	Component Reference
8	The Verification Pipeline (10 Checks)
9	Capability 1 - Attested Inference Provenance
10	Capability 2 - Attested Decision Ledger
11	Capability 3 - Attested Multi-Judge Consensus
12	Capability 4 - Full DCAP on Real Silicon
13	Capability 5 - On-Chain Verification
14	The Tamper Demonstration
15	Security Model and Honest Boundaries
16	Technology Stack
17	Evidence and Live Test Results
18	Completed Work
19	Pending Work
20	Recent Changes - August 2026 Hardening
21	How to Run
A	Appendix A - Repository Inventory
B	Appendix B - API Reference
C	Appendix C - Configuration Reference

1. Executive Summary

Veriform makes AI agents provable. Every decision an agent makes is wrapped in a cryptographic receipt binding that exact decision to unaltered code running inside a hardware-secured enclave. Any third party - an end user, an auditor, or a smart contract - can verify the receipt in seconds and reject anything forged, tampered with, or produced outside the enclave.

The reference application is a wallet-guardian agent that approves or denies crypto transfer requests. The decision engine is deliberately simple, because **the decision is not the product - the receipt is**. The mechanism generalises to any agent whose outputs carry consequences: financial agents, medical triage, compliance review, autonomous on-chain actors.

What is contributed

The TEE primitive already exists. Veriform's contribution is the consumable trust layer above it, and five capabilities that as an integrated, independently verifiable package have no off-the-shelf equivalent:

- **Attested inference provenance** - binds which model judged, under which hashed prompt, and what it actually returned. Catches a backdoored judgment prompt inside a genuine enclave.
- **Attested decision ledger** - a hash-chained history proving completeness, ordering, and a cumulative policy invariant across decisions.
- **Attested multi-judge consensus** - an independently recomputable quorum, so no single rogue or hallucinating model can force a high-stakes approval.
- **Full offline DCAP** - the entire Intel chain of trust verified locally, with no paid service.
- **On-chain verification** - the identical binding recomputed in Solidity and gated on Automata's on-chain DCAP verifier, live on Ethereum Sepolia.

Status at a glance

Dimension	Status
Implementation	Complete and demonstrated end to end; 5 services, ~2,800 lines
Test suite	77 tests passing across 12 files; GitHub Actions CI green
Real hardware	Proven on Intel TDX (Phala Cloud, tdx.small, node prod9 US-WEST-1)
Official simulator	Validated against Phala dstack simulator v0.5.3
On-chain	3 contracts live on Ethereum Sepolia; gate demonstrated
Verification pipeline	10 checks, each failing for a real cryptographic reason
Roadmap	Phases 1-5 complete; 1 unchecked item; Phase 6 is a stretch goal

The honest framing

Veriform introduces no new cryptography. It composes standard primitives - SHA-256, ECDSA over secp256k1 and P-256, X.509 chain validation, quorum logic - into a form that is not otherwise packaged for autonomous agents. The contribution is the integration, its verifiability to outsiders, and its explicit honesty about boundaries.

2. Problem Statement

AI agents increasingly act on their own: approving transactions, handling private data, executing on-chain operations. But every agent runs on infrastructure **someone else controls**. Whoever operates that server can read the agent's inputs, alter its decisions, or extract its private keys - and the end user has no way to detect any of it.

The trust gap has moved up the stack

The hardware primitive that solves the low-level problem already exists and is mature. Providers such as Phala, Marlin and Atoma will run an agent inside a TEE and emit a remote attestation. What is missing is the layer **between the raw attestation and the person or contract who needs to trust it**.

A remote attestation is a multi-kilobyte binary quote containing measurement registers, a certificate chain, and nested signatures. No end user reads one. No smart contract consumes one without help. In practice the user still ends up trusting the operator's claim that the attestation was checked - which returns the problem exactly where it started.

Four concrete failure modes

Attack	What the operator does	Why it is invisible today
Impostor	Runs the agent outside any enclave, serves the same API	Output is byte-identical to the genuine agent
Forgery	Fabricates an attestation quote of the right shape	Structure and signature can be made to look plausible
Substitution	Replays a genuine quote alongside a different decision	The quote really is genuine - it just is not about this decision
Policy swap	Keeps the code identical, swaps the LLM judgment prompt	Code measurement still matches; the enclave is genuinely unaltered

The subtle one

The fourth is the deep case, and the reason attesting code alone is insufficient. If an agent's behaviour is governed by a prompt rather than by its code, then a genuine enclave running unmodified, correctly measured code can still produce attacker-chosen decisions. Any system that attests only the binary declares this attack invisible.

3. Background: TEEs and Remote Attestation

A **Trusted Execution Environment** is a hardware-isolated region of a machine whose memory the host operating system, hypervisor and root user cannot read or modify. Intel TDX - the technology Veriform targets - extends this to an entire virtual machine, so an unmodified Docker container can run inside one.

What a quote actually contains

Remote attestation is how an enclave proves its state to a remote party. The CPU produces a signed *quote* containing measurement registers and a 64-byte caller-supplied field named `report_data`. That field is the hook Veriform uses.

TDX quote v4 layout (byte offsets)

```
[0:48]      header (version, TEE type)
[48:632]    TD report, 584 bytes
    [184:232]    MRTD - measurement of initial code and data
    [568:632]    report_data - 64 caller-supplied bytes
[632:636]   signature section length
[636:700]   ECDSA P-256 signature over header || TD report
[700:764]   attestation public key (raw x || y)
[770:1154]  Quoting Enclave report
[1154:1218] QE report signature, by the PCK certificate
[1218:...]  PCK certificate chain (PEM): PCK -> Platform CA -> Intel Root
```

The chain of trust

A quote is meaningful only if every link is verified. Intel's DCAP scheme chains as follows, and Veriform validates all five links offline:

```
Intel SGX Root CA
  signs -> PCK certificate (platform-specific)
    signs -> Quoting Enclave report
      binds -> attestation key
        signs -> TD report
          carries -> report_data (Veriform's decision binding)
```

Why the last link matters most

The attestation key's signature covers the TD report, which contains `report_data`. This is the one link a software simulator cannot forge: a simulator serves a real captured quote and patches `report_data` in afterwards, which breaks that signature. Veriform's full-DCAP path therefore distinguishes genuine unpatched silicon from a simulator - and the project documents this rather than hiding it.

4. Existing Solutions vs Veriform

Being precise here is part of the contribution. The value is in an integration that is not otherwise available - not in claiming to have invented primitives that already exist.

What already exists

Existing work	What it provides	Gap it leaves
Phala, Marlin, Atoma	Run agents inside TEEs and produce remote attestations. The core primitive - solved.	Emits a raw quote; no consumable verdict, no decision binding
Intel DCAP / Tiber Trust	Standard quote generation, verification tooling and PKI.	Verifies the quote, not what the workload decided
Automata Network	On-chain DCAP verifier - validate a quote inside a smart contract.	Proves the quote is genuine; says nothing about which decision it covers
Confidential inference	TEE-hosted model serving so a provider can attest its own inference. Early and limited.	Attests the model host, not the calling agent's policy or decision
ZK proofs of attestation	Prove possession of a valid quote without revealing it. Active research (zk-DCAP).	Research stage; orthogonal to binding a decision
Transparency logs	Hash-chained, append-only, ordered logs.	Not enclave-bound and not policy-aware

The open gap - where Veriform sits

The primitive works; the consumable trust layer above it does not. No off-the-shelf solution provides, as a single verifiable package:

- a ten-second, human-and-contract-readable verdict derived from a real attestation;
- binding the **judgment policy** - the LLM prompt - not merely the code, so a backdoored prompt inside a genuine enclave is caught;
- a verifiable decision **history** with cross-decision policy invariants (completeness, ordering, and never exceeding a cumulative limit);
- an attested **quorum** of judges, so no single model can force a high-stakes decision;
- a bounded **replay window**, so a captured approval does not stay valid forever.

Direct comparison

Property	Raw TEE platform	Transparency log	Confidential inference	Veriform
Proves code unaltered	Yes	No	Partly	Yes
Binds a specific decision	No	No	No	Yes
Binds the judgment policy	No	No	No	Yes
Verifies model's reported output	No	No	Partly	Yes
Cross-decision invariants	No	Partly	No	Yes
Quorum of judges	No	No	No	Yes
Replay bounded	No	N/A	No	Yes

Property	Raw TEE platform	Transparency log	Confidential inference	Veriform
Consumable verdict in seconds	No	No	No	Yes
On-chain enforceable	Via Automata	No	No	Yes
Offline verification, no paid service	Varies	Yes	No	Yes

5. Solution Approach - The Decision Receipt

Attestation normally proves *this code is running unaltered*. Veriform extends it to prove *this specific decision was made by that unaltered code*.

The binding scheme

```
canonical      = deterministic JSON of the decision payload (sorted keys, no spaces)
decision_hash  = SHA-256(canonical)
binding        = SHA-256(decision_hash || lowercase_ascii(agent_address))
report_data    = binding || 32 zero bytes      -> 64 bytes, placed inside the quote
signature      = secp256k1 (EIP-191) over canonical, by a key derived INSIDE the TEE
```

Because `report_data` is covered by the hardware signature and commits to both the decision hash and the enclave-held key, an attacker cannot alter the decision, replay a quote for a different decision, or sign with a key from outside the enclave. Each of those three attacks breaks a different equation.

The receipt

Field	Content
payload	The decision: request, APPROVE/DENY, method, notes, timestamp, and - when present - the inference, consensus and ledger blocks
address	Ethereum-style address of the enclave-derived key
signature	secp256k1 signature over the canonical payload
quote	TDX attestation quote whose <code>report_data</code> binds the decision

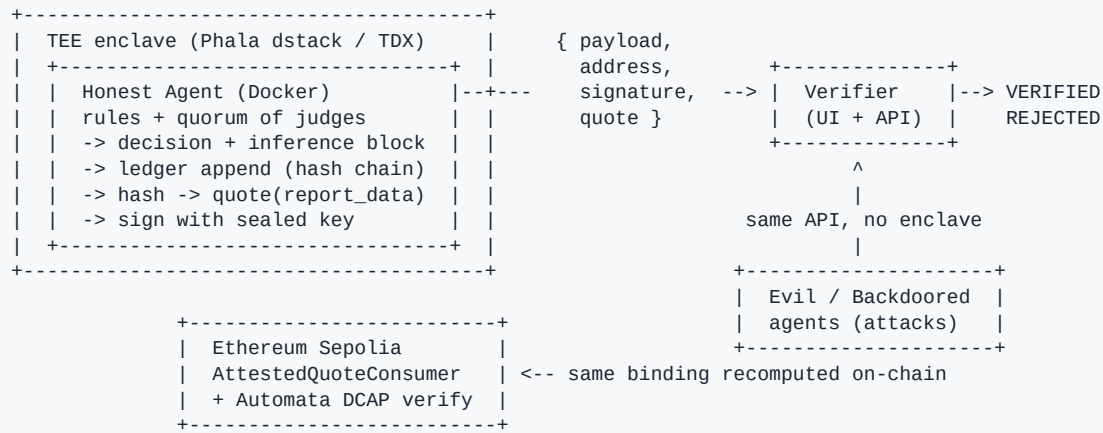
The verifier

The verifier runs **outside** the enclave and trusts nothing but mathematics. It shares no secret with the agent; everything it checks is recomputable from the receipt plus public knowledge. It recomputes the binding, extracts `report_data` from the quote, recovers the signer, validates the Intel chain, and checks the inference, consensus and ledger blocks - rendering one unambiguous verdict with a per-check breakdown a non-expert can read in seconds.

Design principle

Every rejection is for a real cryptographic reason. There is no hardcoded 'if evil' branch anywhere in the verifier. The evil agents are rejected because equations fail, which is what makes the demonstration meaningful rather than theatrical.

6. System Architecture



Services

Service	Port	Role
dev-sim	8090	Local dstack wire-protocol shim; serves a real captured quote and patches report_data. Explicitly not a TEE.
agent	8001	Honest agent: rules, judges, ledger, enclave key and quote
evil-agent	8002	Impostor outside any enclave; no quote or a forged one
backdoored	8003	Genuine enclave path with a swapped judgment prompt
verifier	3000	Verification pipeline, REST API and demo UI

Trust boundary

The agent is inside the boundary and is trusted only insofar as attestation proves it. The verifier is outside and trusts nothing. The operator - server, root user, cloud admin - is explicitly untrusted and is assumed to control the host, the network, and process lifecycle.

7. Component Reference

Module	Lines	Responsibility
agent/app/enclave.py	130	TEE interface. Derives the signing key inside the enclave, requests quotes via dstack or the generic Linux TSM path, computes report_data and signs the canonical payload.
agent/app/decision.py	325	Decision engine: hard rules first, then a pluggable judge (Anthropic / Gemini / Ollama / none), plus multi-judge consensus. Every provider fails closed.
agent/app/ledger.py	96	Append-only hash chain with a cumulative daily-approval accumulator.
agent/app/main.py	83	FastAPI surface; assembles the payload, applies the cross-decision override, attests.
verifier/app/verify.py	560	The 10-check pipeline plus sequence verification for a full decision history.
verifier/app/dcap.py	164	Full Intel DCAP chain validation, entirely offline.
verifier/app/main.py	91	Verifier API: /ask, /verify-sequence, /verify-dcap, and the UI.
evil-agent/app/main.py	52	The adversary: same API, always APPROVE, random key, absent or forged quote.
dev-sim/sim.py	48	dstack wire-protocol shim for local development on Windows.
contracts/*.sol	184	VeriformRegistry, DemoConsumer, AttestedQuoteConsumer.

A deliberate architectural choice

verify_receipt is a pure function. It holds no state, shares no secret, and can be run by anyone against a receipt they were handed. That purity is why an auditor can re-verify the same receipt repeatedly without it failing the second time - and it is the reason certain stateful defences were deliberately left to the caller rather than folded in.

8. The Verification Pipeline (10 Checks)

Each check defends against a specific attack and fails for a specific mathematical reason. Checks report PASS, FAIL, or SKIPPED - and a skip is always stated rather than silently treated as a pass.

#	Check	Defends against	Mechanism
1	quote_present	An impostor outside any enclave	No attestation quote means rejection
2	quote_structure	Malformed or truncated quotes	Must parse as a TDX-quote-sized structure
3	enclave_measurement	Different (backdoored) code in a genuine enclave	MRTD must equal the pinned known-good build
4	decision_binding	Altering the decision; replaying a quote	report_data must equal SHA-256(decision_hash address)
5	signature	Tampered payload; wrong signer	Signature must recover to the attested key
6	freshness	Replaying a genuine, still-valid receipt later	Signed timestamp must be inside the pinned window
7	inference_provenance	Backdoored prompt; agent lying about model output	Prompt hash must match the pinned audited one; action must match the model's actual output
8	ledger_link	Malformed history link; over-limit accumulator	root = SHA-256(prev_root entry_hash); daily_total within limit
9	consensus	A single rogue judge forcing an approval	Verifier recomputes the tally and the quorum rule independently
10	quote_authenticity	Forged quotes with no genuine Intel collateral	PCK chain must root in the pinned Intel SGX Root CA; optionally full DCAP

History-level checks (verify_sequence)

Property	Defends against	Mechanism
Completeness	Silently dropping a decision	A missing link breaks the chain and leaves a sequence gap
Ordering	Reordering or inserting decisions	Chained roots plus monotonic sequence numbers
Policy invariant	Exceeding a cross-decision limit; lying about the running total	The verifier recomputes the accumulator independently

The omission rule

A pinned policy combined with a **missing** block is a failure, never a skip. Without this rule an operator could evade a pinned check simply by not emitting the field it inspects - turning a policy into a suggestion. This rule was added in August 2026 after exactly that downgrade path was found and demonstrated (Section 20).

9. Capability 1 - Attested Inference Provenance

Attesting the code is not enough when the behaviour is governed by a prompt. This capability binds the judgment layer itself into the receipt.

What is bound

Field	Meaning
provider / model	Which model was consulted
system_prompt_sha256	SHA-256 of the judgment criteria actually used
input	The exact request put to the model
output	What the model actually returned - action and reason

Two distinct attacks caught

- **Backdoored prompt.** The verifier pins the SHA-256 of the audited prompt. A genuine enclave running unaltered code but a swapped prompt produces a different hash and is rejected.
- **Misreported output.** The top-level action must equal what the model returned. An agent claiming APPROVE while the model returned DENY is rejected.

Demonstrated live

With a real Gemini judge configured, the honest agent's receipt carries prompt hash d4bac6b7... matching the pinned value, while the backdoored agent carries 9ef1055b... and is REJECTED - with quote, MRTD, binding and signature all still passing. The enclave is genuine; only the policy was swapped.

Honest boundary

This proves which model was consulted, under which criteria, and that the agent reported the result faithfully. It does **not** prove the provider's servers ran that model unmodified - that requires the provider to return its own attestation. The inference block is designed to carry a provider attestation when one becomes available.

10. Capability 2 - Attested Decision Ledger

A single receipt proves one decision. A *history* requires proving that nothing was dropped, reordered, or inserted - and that a rule spanning decisions was never violated.

```

root_0      = 32 zero bytes
entry_hash_n = SHA-256(canonical(decision_core_n))    # payload minus its own ledger block
root_n      = SHA-256(root_(n-1) || entry_hash_n)

```

Because the whole ledger block is inside the signed, quote-bound payload, an outsider can verify a sequence is complete, correctly ordered, and policy-compliant - recomputing the cumulative daily-approval total independently rather than trusting the agent's claim.

Honest boundary

This is tamper-evident within a presented sequence. A malicious operator could still roll the enclave back and present a shorter or forked history. Defeating that needs the latest root anchored outside the enclave - a hardware monotonic counter, or periodically posting the root on-chain. The ledger is structured so this anchoring is a small addition.

11. Capability 3 - Attested Multi-Judge Consensus

A single LLM can hallucinate, be jailbroken, or be quietly swapped. Consensus makes 'no single judge can approve alone' a property an outsider can check.

Three deliberately different judges vote: an LLM judge, a scam and urgency-language heuristic, and a conservative amount heuristic. Every vote is bound into the receipt.

The verifier independently...	Which defeats
Recomputes the approval tally from the votes	An agent misreporting the count
Confirms the final action follows the quorum rule	A rogue judge forcing an approval
Enforces the pinned threshold and judge count	An agent lowering its own bar
Rejects an APPROVE with no consensus block when a quorum is pinned	Evading the pin by omission

12. Capability 4 - Full DCAP on Real Silicon

The certificate-chain check proves a quote carries genuine Intel collateral. Full DCAP goes the whole way, verifying every signature from Intel's root down to report_data - offline, with no paid service.

Link	What it proves
quote_format	A genuine TDX v4 quote structure
att_key_signs_report	The hardware signed this TD report over our report_data
qe_binds_att_key	The Quoting Enclave report commits to that attestation key
pck_signs_qe	Intel's platform certificate signed the QE report
chain_to_intel_root	The chain roots in the pinned Intel SGX Root CA

Documented rather than hidden

Full DCAP correctly FAILS on a simulator-served quote, because a simulator patches report_data after the hardware signed it. The project states this openly in dcap.py: it is precisely the guarantee real silicon adds, and the check is off by default so the simulator demo still runs honestly.

13. Capability 5 - On-Chain Verification

Two independent on-chain guarantees: the **signature** path (was the enclave key the signer?) and the **hardware** path (was the quote genuine Intel TDX?).

Contract	Address on Ethereum Sepolia	Role
VeriformRegistry	0x5d0CeA3564a959959C04B4A9dDc0D6209ED3937C	Stores the attested agent key; verifies decision signatures via ecrecover
DemoConsumer	0x03Fb74db45e3a6282107393be73fC7D12147C6B2	A contract action gated on 'signed by a verified enclave'
AttestedQuoteConsumer	0x6658DD646FaCc994bc358183F2c39FFfd25068b8	On-chain DCAP: gates an action on Automata accepting the quote AND report_data matching the decision
Automata DCAP verifier	0x63eF330eAaadA189861144FCbc9176dae41A5BAf	Third-party on-chain attestation entrypoint (audited by Trail of Bits)

The mirror

AttestedQuoteConsumer recomputes the identical binding in Solidity, including a hand-written `_toLowerHexBytes` that reproduces exactly the ASCII bytes Python's `address.lower().encode()` produces. Six tests prove the on-chain extraction and recomputation agree with both the enclave and the off-chain verifier against a real hardware quote.

Gate demonstrated on a public chain

A genuine enclave signature executes (`approvedCount = 1`). The evil agent's signature reverts with `UnverifiedDecision`. Bytecode confirmed for all three contracts.

14. The Tamper Demonstration

Four agents, one API, four different checks catching four different attacks. Nothing is hardcoded - each rejection is an equation that does not balance.

Agent	Verdict	Caught by	Why
Honest	VERIFIED	-	All checks pass
Evil (no quote)	REJECTED	quote_present	Runs outside any enclave
Evil (forged quote)	REJECTED	decision_binding	Quote is structurally plausible but cannot commit to the decision
Backdoored prompt	REJECTED	inference_provenance	Genuine enclave, valid quote and signature - but the judgment prompt was swapped

The forged-quote case is the instructive one: it passes quote-present, structure and signature, and is still caught, because `report_data` cannot commit to a decision that was not made inside the enclave.

15. Security Model and Honest Boundaries

Trust model

Party or component	Trusted?	Notes
Intel (hardware and PKI root)	Yes - root of trust	The Intel SGX Root CA public key is pinned
The TEE / enclave	Only if attested	Trusted insofar as a valid quote proves unaltered code
The enclave-derived key	Yes	Derived inside the TEE, never leaves it
The operator / host	No	Can read or alter anything outside the enclave, control the network, restart or roll back
The agent's own code	Only via measurement	Pin EXPECTED_MRTD; a different build is rejected
The LLM judgment prompt	Only via hash	Pin the audited prompt hash; a swap is rejected
The remote LLM provider	No	Attested only if the provider returns its own attestation
The verifier	Trusts only mathematics	Shares no secret; everything is recomputable

What Veriform does NOT protect - stated deliberately

Boundary	Status	Intended mitigation
Liveness / denial of service	Out of scope by design	An operator can always refuse to run the agent; availability is not the claim
Confidentiality of inputs	Out of scope by design	Encrypting inputs to the enclave is a natural extension
Remote LLM execution	Not provable today	Requires provider-side confidential inference attestation
Ledger rollback / forking	Open	Anchor the latest root outside the enclave - monotonic counter or on-chain
Replay within the window	Bounded, not eliminated	Age cap implemented Aug 2026; single-use needs caller-side state or a nonce
Enclave clock trust	Open	Host-influenced; a trusted in-enclave time source would close it
Automated key rotation	Open	Supported in principle; not implemented
Side channels / TEE breaks	Out of scope	Vendor domain; multi-vendor attestation is the intended answer

Why this section exists

A verification system that overstates its guarantees is worse than one that states them exactly. Every boundary above is published in SECURITY.md alongside the defences, and two of them were revised in August 2026 to be more accurate rather than more flattering.

16. Technology Stack

Layer	Technology	Role
Enclave runtime	Phala dstack, Intel TDX	Full-VM isolation; runs unmodified Docker containers
Quote backends	dstack SDK; Linux TSM configs	TSM path works on any Intel TDX VM with kernel 6.7+, no dstack
Services	Python 3.12, FastAPI, uvicorn, pydantic	Agent, evil agent, verifier, dev shim
Signing	eth-account, secp256k1 (EIP-191)	ecrecover-compatible, so the on-chain path is trivial
Attestation crypto	cryptography (X.509, ECDSA P-256)	Offline full-DCAP chain validation
Judgment layer	Anthropic Claude, Google Gemini, Ollama	Pluggable; all providers fail closed
Smart contracts	Solidity 0.8.24, py-solc-x, web3.py, eth-tester	Registry, consumer, on-chain DCAP gate
On-chain attestation	Automata DCAP verifier	Full Intel chain enforced inside the EVM
Verifier UI	Static HTML, CSS and vanilla JavaScript	Served by FastAPI; no build step
Packaging	Docker, docker-compose	One compose file per environment
CI	GitHub Actions, pytest	77 tests on every push and pull request

Code inventory

Component	Lines	Files
verifier/app	726	4
tests	750	13
agent/app	521	5
scripts	511	7
contracts	184	3
evil-agent/app	52	2
dev-sim	48	1
Total	~2,792	35

17. Evidence and Live Test Results

Phase 1 - Official Phala dstack simulator (v0.5.3)

The simulator ships no Windows binary, so the Linux musl build runs inside a ~3.5 MB Alpine WSL2 distro reachable over TCP. The honest agent verifies with a real 5,006-byte TDX quote: quote_present, quote_structure, enclave_measurement with the real MRTD pinned, decision_binding and signature all pass. Only quote_authenticity is skipped, which needs Intel PKI. This proved the binding scheme and byte offsets against the genuine dstack quote format.

Phase 3 - Real Intel TDX silicon

Parameter	Value
Platform	Phala Cloud, Intel TDX, tdx.small, node prod9 US-WEST-1
CVM UUID	54b4c81d-6802-4058-9961-5dff7a64189e
App ID	5fa4f8951a132c6eefff967a8227aca90e8dc0c4
Enclave address	0xEEeba6E20b068A68eCFCB4f95128b2B346D39929
Quote size	5,010 bytes, TDX v4
Result	All five DCAP links plus decision binding PASS

Critically this includes att_key_signs_report - the link only real, unpatched silicon can satisfy. The quote is preserved as a regression fixture with a dedicated test, so the guarantee cannot silently regress.

Test suite - 77 tests, 12 files

File	Tests	Covers
test_freshness.py	10	Replay window: stale, boundary, missing, malformed, future-dated, skew, history exemption
test_verify.py	10	End-to-end verdicts and MRTD pinning
test_decision.py	9	Rule engine and fail-closed behaviour
test_consensus.py	8	Quorum, rogue judge, misreported tally, pin evasion
test_ledger.py	8	Dropped, reordered, tampered entries; lying accumulator
test_inference.py	7	Backdoored prompt, misreported model output, omission
test_automata.py	6	Solidity binding equivalence, byte for byte
test_dcap.py	5	Full chain on real quote; tampered report and QE report
test_tsm.py	5	Generic Linux TDX backend
test_intel_chain.py	3	Intel PKI chain validation
test_onchain.py	3	ecrecover compatibility
test_phase3_live.py	3	Live hardware quote regression

18. Completed Work

Core system

Item	Status
Problem statement, architecture, threat model	Complete
Agent: decision -> hash -> quote -> signature	Complete
Verifier with 10-check pipeline and per-check breakdown	Complete
Demo UI with four agent modes	Complete
Tamper demonstration: impostor, forger, backdoored prompt	Complete
Pluggable judgment layer, all providers failing closed	Complete
Attested inference provenance	Complete
Attested decision ledger with cross-decision invariant	Complete
Attested multi-judge consensus	Complete
Full offline DCAP verification	Complete
Replay window bounding	Complete (Aug 2026)

Validation milestones

Phase	Outcome
Phase 1 - Official simulator	Validated against Phala dstack v0.5.3 via WSL2; proof archived
Phase 2 - LLM judgment live	Gemini free tier: reasoned APPROVE and DENY inside verified receipts
Phase 3 - Real TDX silicon	Genuine 5,010-byte quote; all five DCAP links pass; fixture and test archived
Phase 4 - Demo polish	Forged-quote toggle, 2-minute demo script with judge Q&A;., screenshots
Phase 5 - On-chain anchor	Three contracts live on Ethereum Sepolia; gate demonstrated publicly

Reproducibility paths

- **Zero-cost path** - `scripts/tdx-quote.sh` on any Intel TDX VM using the generic Linux TSM interface. No Phala account, no dstack, no Docker.
- **Phala Cloud path** - `scripts/deploy-tdx.sh` deploys, captures a real quote, and tears down.
- **Helper path** - `docs/PHASE3-HELPER.md` is a single self-checking block anyone with TDX hardware can run and return; the proof is self-standing.
- **Local path** - `scripts/run-local.ps1` runs the full five-service demo on Windows with no Docker.

19. Pending Work

Nothing in the system is broken or half-built. What follows is genuinely outstanding, ranked by value.

1. Automata PCCS collateral upsert

The only unchecked roadmap item

AttestedQuoteConsumer is deployed and its logic is proven equivalent to the enclave and off-chain verifier. Fully exercising `verifyAndAttestOnChain` against the live quote additionally requires that platform's collateral to be present in Automata's on-chain PCCS. This is a data-registration step, not an engineering one.

2. Ledger rollback anchoring

The decision ledger is tamper-evident within a presented sequence but does not prevent an operator rolling back and presenting a shorter or forked history. The fix is to anchor the latest root outside the enclave - a hardware monotonic counter, or periodically posting the root on-chain. The ledger was designed so this is a small addition, and the Sepolia deployment already provides a natural anchor point.

3. Single-use receipts

The August 2026 freshness check bounds the replay window but does not make a receipt single-use: inside the window it can still be re-presented. Closing this requires caller-side state - remembering (address, ledger sequence) - or a relying-party nonce. It was deliberately left out of `verify_receipt` to preserve that function's purity, since an auditor must be able to re-verify the same receipt without it failing.

4. Trusted time inside the enclave

The enclave clock is host-influenced, so a future-dated receipt is only caught beyond the skew tolerance. A trusted in-enclave time source would close this.

5. Verify the MRTD coverage assumption

An open question, not a defect

Several defences assume the pinned MRTD covers what the operator can change. If `dstack`'s compose configuration - and therefore environment variables such as `JUDGE_PROVIDER` and `TRUSTED_ADDRESSES` - is measured, then some attacks require a stronger adversary than assumed. If it is not, the environment is an operator-controlled input to a trusted computation. This determines how the residual risk should be described in `SECURITY.md` and needs deployment knowledge rather than code reading.

6. Automated key rotation

Key rotation under fresh attestation is supported in principle; automation is not implemented.

7. Phase 6 - multi-vendor attestation (stretch)

Requiring agreement across vendor roots - Intel, AMD, Nvidia - so that no single PKI compromise breaks the guarantee. Genuinely new work, explicitly a stretch goal.

8. Documentation discrepancies

Location	Says	Actually
README stack table	Verifier is a React page	Static HTML, CSS and vanilla JavaScript served by FastAPI

Location	Says	Actually
README stack table	On-chain contract on Base Sepolia	Deployed to Ethereum Sepolia

9. Minor engineering

- `Enclave()` is constructed per request in `agent/app/main.py`, re-deriving the signing key from `dstack` on every decision. Caching it removes an avoidable round trip from the hot path.
- `quote_authenticity` is skipped once an earlier check has already failed. The verdict is correct, but the receipt shows less than it could.

20. Recent Changes - August 2026 Hardening

Two security defects were found by audit, proven exploitable against a real Intel quote, fixed, and merged. Both were inside the project's stated threat model and neither was listed among its documented boundaries.

Finding 1 - Pinned checks evadable by omission

Detail	
Defect	inference_provenance and consensus returned SKIPPED when their block was absent, even with the corresponding policy pinned. Because the pipeline hard-fails only on FAIL, a missing block bypassed the pin.
Exploit	An operator sets JUDGE_PROVIDER=none and lists their own address in TRUSTED_ADDRESSES, producing a rules-only APPROVE carrying neither block. Genuine quote, correct binding, valid signature - accepted as VERIFIED.
Notable	The behaviour was specified by a test that set the pin and asserted the skip, which is why the suite was green over it.
Fix	With a pin configured, a missing block is a hard failure - scoped to APPROVE, since a rule-based denial approved nothing and failing it would reject legitimate receipts.
Proof	Identical receipt, real Intel quote: VERIFIED before, REJECTED after.

Finding 2 - Receipts valid indefinitely

Detail	
Defect	SECURITY.md listed replay as a known, not-yet-implemented gap. A captured APPROVE verified cleanly forever.
Insight	The timestamp is already inside the signed, quote-bound payload, so a replayer can re-present an old receipt but cannot re-date it. An age cap is therefore sufficient - no nonce protocol required.
Fix	A freshness check gated on MAX_RECEIPT_AGE_SECONDS, with future-dating rejected beyond a skew tolerance. Not applied to historical entries in verify_sequence, which are retrospective by design.
Proof	One receipt dated two hours ago: VERIFIED with no expiry pinned, REJECTED (7,200s old, past the 300s limit) with one.

Net effect

Metric	Before	After
Verification checks	9	10
Tests passing	63	77
Documented defences claimed but not delivered	2	0

Both fixes left SECURITY.md more accurate rather than more flattering: the first added a defence the document was already claiming, and the second downgraded replay from 'gap' to 'bounded, not eliminated' with both residual limits named explicitly.

21. How to Run

Local demo - Windows, no Docker

```
powershell -ExecutionPolicy Bypass -File scripts\run-local.ps1
# then open http://localhost:3000

# optional: enable the LLM judge and the backdoored-prompt demo
copy .env.example .env      # add GEMINI_API_KEY (free: aistudio.google.com/apikey)
```

Docker

```
docker compose up --build      # agent, evil agent, verifier
pytest tests/ -v                # 77 tests
```

Real Intel TDX hardware, zero cost

```
# on any Intel TDX guest (GCP c3-standard, Azure DCesv6, kernel >= 6.7)
git clone https://github.com/Sakthi-Sundaram-R/Veriform.git && cd Veriform
bash scripts/tdx-quote.sh
```

Appendix A - Repository Inventory

Path	Contents
agent/	Honest agent: enclave.py, decision.py, ledger.py, main.py, Dockerfile
verifier/	Verifier: verify.py, dcap.py, main.py, static UI, Dockerfile
evil-agent/	Adversary service
contracts/	VeriformRegistry.sol, DemoConsumer.sol, AttestedQuoteConsumer.sol
dev-sim/	dstack wire-protocol shim and base quote
scripts/	run-local.ps1, tdx-quote.sh, deploy-tdx.sh, deploy_onchain.py, automata_verify.py, run-official-sim-windows.sh
tests/	12 test files plus real-quote fixtures
docs/	Guides, proofs, screenshots, pitch deck, this report
SECURITY.md	Threat model, per-check defence map, honest boundaries
DEMO.md	Two-minute demo script with judge Q&A;

Appendix B - API Reference

Endpoint	Service	Purpose
POST /decide	agent :8001	Returns payload, address, signature, quote
GET /health	agent :8001	Liveness
POST /ask	verifier :3000	Asks a chosen agent and verifies the receipt
POST /verify-sequence	verifier :3000	Verifies a full decision history
POST /verify-dcap	verifier :3000	Full Intel DCAP verification of a raw quote
GET /	verifier :3000	Demo UI

Appendix C - Configuration Reference

Variable	Applies to	Effect
JUDGE_PROVIDER	agent	auto anthropic gemini ollama none. All fail closed.
CONSENSUS, CONSENSUS_THRESHOLD	agent	Enable quorum judging and set the bar
ROGUE_JUDGE	agent	Demo only: name a judge to compromise
MAX_TX_AMOUNT, TRUSTED_ADDRESSES, TRUSTED_AUTO_LIMIT	agent	Hard rule limits enforced inside the enclave
DAILY_LIMIT	agent	Cumulative daily approval cap enforced by the ledger
QUOTE_BACKEND	agent	dstack (default) or tsm for any Intel TDX VM
EXPECTED_MRTD	verifier	Pin the known-good enclave measurement

Variable	Applies to	Effect
EXPECTED_SYSTEM_PROMPT_SHA256	verifier	Pin the audited judgment prompt
EXPECTED_CONSENSUS_THRESHOLD / _TOTAL	verifier	Pin the required quorum
MAX_RECEIPT_AGE_SECONDS	verifier	Bound the replay window; unset means valid indefinitely
FUTURE_SKEW_TOLERANCE_SECONDS	verifier	Clock-drift allowance before a receipt is pre-minted
FULL_DCAP	verifier	Verify the quote-body signature, not just the cert chain

Veriform is a research and portfolio project. It composes standard primitives into a consumable verification layer; it introduces no new cryptography and has not undergone formal security review. The guarantees stated here are design intent supported by tests and live evidence, not a certified security product.